

Колоночные базы данных на примере СУБД Cassandra

Постреляционные базы данных
Лекция №3

Виноградова М.В.
МГТУ им. Н.Э. Баумана

Колоночные БД – семейство столбцов

- Cassandra, HBase, Amazon SimpleDB, Amazon DinamoDB, HyperTable
- Сервер - кластер
- БД - пространство ключей
- Таблица - **семейство столбцов**
- Столбец - **столбец**

Модель данных

строка: id || имя:значение

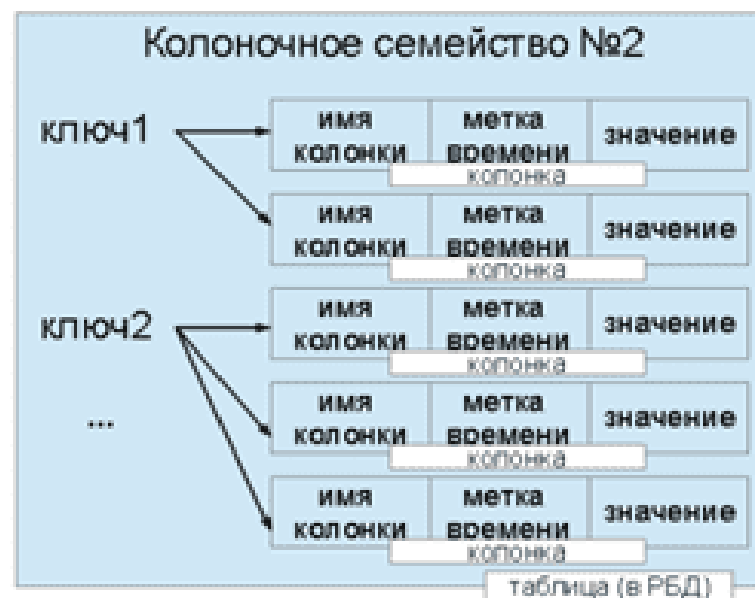
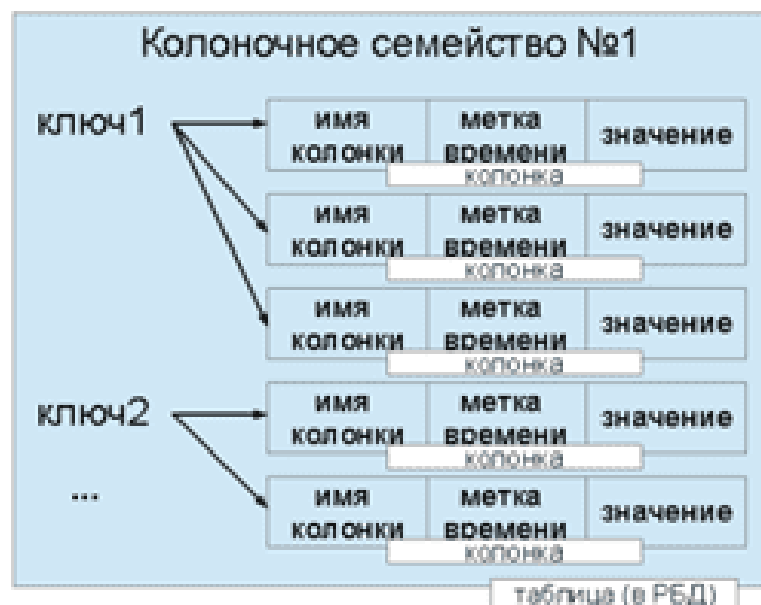
базовый элемент в Cassandra (ключ-значение)

```
{  
    name: " ",  
    value: " ",  
    timestamp: 123456  
}
```

*время жизни (метка времени
конфликт/обновление/удаление)*

Схема модели данных

Пространство ключей схема (в РБД)



Столбцы

```
{  
  {  
    "id": {  
      столбец_1: «значение 1 »,  
      столбец_2: «значение 2 »,  
      ...      }  
    }  
  }  
  {  
    "id2": {  
      столбец_1: «значение 1 »,  
      столбец_3: «значение 3 »,  
      ...      }  
    }  
  }  
}
```

- различные наборы столбцов для разных строк
- могут быть добавлены в процессе
- столбец добавляется/удаляется в любой момент времени

Супер-столбец

- ассоциативный массив

```
{  
  name: "bookA",  
  value: {  
    author: "автор",  
    izd: "название",  
    isbn: 12345,  
  }  
}
```

Строки

- коллекция столбцов
- ассоциируется с ключом

```
"ann": -- ключ строки  
{  
    name: "...",  
    adr: "...",  
    age: "..."  
}
```

Типы данных в СУБД Cassandra

- **ByteType**: любые байтовые строки (без валидации);
- **AsciiType**: ASCII строка;
- **UTF8Type**: UTF-8 строка;
- **IntegerType**: число с произвольным размером;
- **Int32Type**: 4-байтовое число;
- **LongType**: 8-байтовое число;
- **UUIDType**: UUID 1-ого или 4-ого типа;
- **TimeUUIDType**: UUID 1-ого типа;
- **DateType**: 8-байтовое значение метки времени;
- **BooleanType**: два значения: true = 1 или false = 0;
- **FloatType**: 4-байтовое число с плавающей запятой;
- **DoubleType**: 8-байтовое число с плавающей запятой;
- **DecimalType**: число с произвольным размером и плавающей запятой;
- **CounterColumnType**: 8-байтовый счётчик

Пространство ключей

- Пространство ключей (аналог БД), содержит семействе столбцов для ПО

```
CREATE KEYSPACE [IF NOT EXISTS]
```

```
  keyspace_name
```

```
  WITH REPLICATION = {
```

```
    'class' : 'SimpleStrategy', 'replication_factor' : N }
```

```
  | 'class' : 'NetworkTopologyStrategy',
```

```
    'dc1_name' : N [, ...]
```

```
};
```

Работа через клиента БД

- **set** Pers['id']['fio'] = 'значение';
set Pers['id']['fio'] **with** ttl
(ttl – время жизни - после N сек удаляется)
- **get** Pers['id'];
get Pers['id']['fio'];
get Pers **where** fio = '..';
- **del** Pers['id']['fio'];
del Pers['id'];

Язык CQL – Cassandra Query Language

- **create columnfamily** Pers
(
 key varchar **primary key**,
 fio varcahr,
 ...
);

insert into Pers(**key**, fio, ...) **values** ('..', '..', ...);

select * from Pers **where** fio = "
 (после create index)

• **update** Pers **set** age = age + 1 **where** key = '..';

Поиск

- По умолчанию поиск только по ключу строки
- или надо явно указать индекс, тогда возможен поиск по значениям индексам
- битовые индексы, которые эффективны при небольшом значении (количестве) столбцов

Распределительный и кластерный ключи

```
CREATE TABLE numberOfRequests  
( cluster text,  
  date text,  
  time text,  
  numberOfRequests int,  
  PRIMARY KEY ((cluster, date), time)  
)  
WITH default_time_to_live = 120 and  
  CLUSTERING ORDER BY (time DESC)
```

- Распределительный – узел хранения
- Кластерный – внутри узла

Поисковые запросы (секция WHERE)

- Столбцы **распределительного** ключа (указывать все) поддерживают только два оператора: = и IN

```
SELECT * FROM numberOfRequests WHERE cluster IN ('cluster1', 'cluster2') AND date = '2015-05-06';
```
- **Кластерные** ключи поддерживают операторы =, IN, >, > =, <=, <, CONTAINS и CONTAINS KEY
- Выборки по **индексным** полям
- Выборка по прочим полям – полное чтение данных

```
SELECT * FROM student  
WHERE last_name='Ivanov' ALLOW FILTERING;
```

Возможности CQL

- Агрегация, группировка и сортировка
SELECT title, MAX(price)
FROM course GROUP BY title;
- Материализованные представления
- Пакетная обработка
- Время жизни - TTL
- Конструкция if
UPDATE course SET description='conditional'
WHERE title='test6' AND price=2000 **IF EXISTS**;

Масштабирование в СУБД Cassandra

- **горизонтальное** - добавление узлов, без остановки кластера
- увеличение емкости
- увеличение количества операций чтения/записи

Доступность

- Одноранговая репликация
- доступность, согласованность (R, W при фиксированном $N = 10$)
 - $N = 3, R = 2, W = 2$ - если 1 узел сбой, то действие получит от другого
 - $R = 1, W = 2$ - если 2 узла сбой, то нет записи, но есть чтение
 - $R = 2, W = 1$ - если 2 узла сбой, то есть запись, но нет чтение
 - $(R + W) > N \Rightarrow$ выбор согласованности/доступности

Схема чтения



Уровни согласованности чтения

- **ONE** — координатор шлёт запросы к ближайшему узлу-реплике, читая остальные с целью исправления (read repair) с заранее заданной в конфигурации вероятностью;
- **TWO** — координатор шлёт запросы к двум ближайшим узлам, выбирая значение с большей меткой времени;
- **THREE** — координатор шлёт запросы к трем ближайшим узлам, выбирая значение с большей меткой времени;
- **QUORUM** — собирается кворум, то есть координатор шлёт запросы к более чем половине узлов-реplik, а именно $\text{round}(N/2)+1$, где N — уровень репликации;
- **ALL** — координатор возвращает данные после прочтения со всех узлов-реplik;
- **LOCAL_QUORUM** — собирается кворум узлов в том датацентре, где происходит координация, и возвращаются данные с последней меткой времени;
- **EACH_QUORUM** — координатор возвращает данные после собрания кворума в каждом из датацентров.

Схема записи



Уровни согласованности записи

- **ANY** — даёт возможность записать данные, даже если все узлы-реплики не отвечают. Координатор дожидается первого ответа от любого одного узла-реплик или данные сохраняются с помощью механизма направленной отправки (hinted handoff) на координаторе.
- **ONE** — координатор шлёт запросы всем узлам-реплик, но возвращает управление пользователю, дождавшись подтверждения от любого первого узла;
- **TWO** — координатор дожидается подтверждения от двух первых узлов, прежде чем вернуть управление;
- **THREE** — координатор ждет подтверждения от трех первых узлов, прежде чем вернуть управление;
- **QUORUM** — координатор дожидается подтверждения записи от более чем половины узлов-реплик, а именно $\text{round}(N/2)+1$, где N — уровень репликации;
- **ALL** — координатор дожидается подтверждения от всех узлов-реплик;
- **LOCAL_QUORUM** — координатор дожидается подтверждения от более чем половины узлов-реплик в том же центре обработки данных, где расположен координатор. Это позволяет избавиться от задержек, связанных с пересылкой данных в другие датацентры.
- **EACH_QUORUM** — координатор дожидается подтверждения от более чем половины узлов-реплик в каждом центре обработки данных.

Уровни согласованности сравнение

- **One**
 - Чтение – быстро/устаревание
 - Запись – быстро/потеря данных
- **Quorum**
 - Чтение – новое вр/чт с исправлением
 - Запись – подтверждение записи
- **ALL**
 - При отказе блокировка
- По умолчанию для всех операций **ONE**
- Задается через параметр **CONSISTENCY**

Применение колоночных БД

- регистрация событий
- управление информацией блогов
- счетчики посещений
- сроки действия (реклама, новости)
- Не используется:
 - Транзакции чтения/записи
 - агрегация (только на клиенте)
 - для раннего проектирования

Спасибо за внимание!

Виноградова Мария Валерьевна

Vinogradova.m@bmstu.ru

МГТУ им. Н.Э. Баумана
кафедра Систем обработки информации и
управления (ИУ5)